Data in R

Today's agenda:

Discuss loading data, data types, etc.,

Pointers on data

Load and examine some data

Electronic Formats

R can handle most common data formats

E.g., .csv, .txt

Proprietary data formats may require additional packages

E.g., Excel .xlsx files, ESRI .shp files

R has its own file types:

E.g., .RDS

Different pros/cons to data formats!

Why plain text?

- Readable by anything, on any machine
- Won't become obsolete with the next software version
- Survive partial corruption
- Note: A .csv is just a text file that tells you how to parse it
  - csv = comma separated value

We practise this

Your assignments: .R, .Rmd, .Qmd

This course's syllabus: Markdown

The data in data/: .csv

- These are all basically “text” files
- **Not an accident!**

Getting data into R

- Functions used will depend on data types
- Often multiple options for a given file type
- Functions can mess things up when importing them!
- R can pull from different places (online, locally, databases, etc.)

Getting data into R: example functions

File type	Extension(s)	Common R import function(s)	Package(s)
Comma-separated values	`.csv`	`read.csv()`, `read_csv()`	base, readr
Tab-delimited text	`.tsv`, `.txt`	`read.delim()`, `read_tsv()`	base, readr
Excel (old)	`.xls`	`read.xls()`, `read_excel()`	gdata, readxl
Excel (modern)	`.xlsx`	`read.xlsx()`, `read_excel()`	openxlsx, readxl
R object (binary)	`.RData`, `.rda`	`load()`	base
R object (single)	`.RDS`	`readRDS()`	base
SPSS	`.sav`	`read.spss()`, `read_sav()`	foreign, haven
Stata	`.dta`	`read.dta()`, `read_dta()`	foreign, haven
SAS	`.sas7bdat`, `.xpt`	`read_sas()`, `read.xport()`	haven, sas7bdat, foreign
JSON	`.json`	`fromJSON()`	jsonlite
XML	`.xml`	`xmlParse()`, `read_xml()`	XML, xml2
Feather/Arrow	`.feather`, `.arrow`	`read_feather()`, `read_parquet()`	arrow
HDF5	`.h5`	`h5read()`	rhdf5
NetCDF	`.nc`	`nc_open()`	ncdf4
SQLite database	`.sqlite`, `.db`	`dbConnect(RSQLite::SQLite(), ...)`	RSQLite
Shapefile (GIS)	`.shp`	`st_read()`	sf
GeoPackage	`.gpkg`	`st_read()`	sf

Getting data into R: example

1. Go to the course Github site https://github.com/bmaitner/biometry_course
2. Navigate to **data/Avonet**
3. Download the file "**AVONET1_BirdLife.csv**"

Getting data into R: example

- Since the file “**AVONET1_BirdLife.csv**” is a csv, we’ll use **`read.csv()`**

- We can do this using
  - Absolute paths
  - Relative paths
  - A remote copy of the file

Getting data into R: relative vs absolute paths

Absolute vs relative paths

- Absolute: path from the root of your file system
- Relative: path relative to your working directory

Getting data into R: relative vs absolute paths

Absolute vs relative paths

- Absolute: path from the root of your file system
- Relative: path relative to your working directory

Examples: which is which?

“C:/Users/bmaitner/Desktop/current_projects/biometry_course/data/Avonet/AVONET1_BirdLife.csv”

“data/Avonet/AVONET1_BirdLife.csv”

Getting data into R: relative vs absolute paths

Try to read in the file you downloaded (AVONET1_BirdLife.csv)

- Try reading it using both relative and absolute paths
- Use **`read.csv()`**
- Tab completion makes it easier
  - **`read.csv("")` then hit the TAB key**
    - **Cursor needs to be between the ""**

Getting data into R: remote data

Many R functions can also read in online data

```
avonet <- read.csv("data/Avonet/AVONET1_BirdLife.csv")
```

```
avonet_v3 <-
read.csv("https://github.com/bmaitner/biometry_course/raw/refs/heads/main/data/Avonet/AVONET1_BirdLife.csv")
```

```
identical(avonet, avonet_v3)
```

Same data, two routes.

Not everything is a .csv

Try this (download if needed):

```
amphibians <- read.csv(
  "data/European_amphibians/oo_32985.xlsx")
```

What happens? Why?

That's not text

Warning message:

> line 1 appears to contain embedded nulls
>
> .xlsx isn't text. It's a zip file full of XML.

`read.csv()` reads ***text***. It tried.

You need a package

Base R can't read Excel files. Someone else wrote code that can.

```
install.packages("readxl")
```

Needed once per computer. Not once per session.

Installing is not loading

Try:

```
read_xlsx("data/European_amphibians/oo_32985.xlsx")
```

Error: could not find function "read_xlsx"

Installed ≠ available.

Run:

```
library(readxl)
```

Now read it

```
library(readxl)

amphibians <- read_xlsx(

  "data/European_amphibians/oo_32985.xlsx")

head(amphibians)
```

Did it work?

Look at the file first

Rows 1-3: various metadata
Row 4: the actual column names

	A	B	C	D	E	F	G	H	I	J	K	L
1												
2	**Traits**	Sexual dimorphism	Body mass (in g)				Body length (in mm)					
3							Snout-to-vent length (in mm)			Total length (in mm)		
4	**Species**	Presence of sexual dimorphism	Body mass in males	Body mass in females	Adult body mass	Body mass in juveniles	Snout-to-vent length in males	Snout-to-vent length in females	Adult snout-to-vent length	Total length in males	Total lenght in females	Adult total length
5	*Alytes cisternasii*	0	DD	DD	5.02	DD	36.75	38.38	37.56	37.75	39.83	37.56
6	*Alytes dickhilleni*	DD	DD	DD	DD	DD	44.70	DD	47.17	DD	DD	47.17
7	*Alytes muletensis*	0	DD	DD	2.31	DD	34.70	35.65	35.18	DD	DD	35.18
8	*Alytes obstetricans*	0	DD	11.70	11.70	3.50	45.05	49.57	47.31	46.15	51.60	47.31
9	*Anaxyrus americanus*	DD	118.70	59.30	60.00	DD	62.87	76.89	69.88	64.56	76.89	69.88
10	*Atylodes genei*	1	DD	DD	3.92	DD	53.29	49.00	51.15	DD	DD	98.24
11	*Bombina bombina*	1	3.17	2.97	3.07	0.87	38.89	40.00	39.44	36.60	36.80	39.44
12	*Bombina pachypus*	DD	DD	DD	DD	DD	45.75	45.25	45.50	DD	DD	45.50
13	*Bombina variegata*	1	6.56	6.77	6.32	DD	45.08	44.34	44.71	DD	DD	44.71
14	*Bufo bufo*	1	41.50	164.75	76.54	DD	66.11	88.46	77.28	62.73	83.17	77.28
15	*Bufo mauritanicus*	DD	DD	DD	DD	DD	132.00	150.00	141.00	DD	DD	141.00
16	*Calotriton arnoldi*	DD	DD	DD	DD	DD	58.95	58.45	58.70	104.60	104.60	103.95
17	*Calotriton asper*	1	7.10	6.13	6.62	1.80	64.97	61.67	63.32	112.93	121.01	116.70
18	*Chioglossa lusitanica*	1	DD	DD	2.00	DD	45.09	45.94	45.51	156.00	164.00	159.00
19	*Discoglossus galganoi*	1	21.80	15.20	18.50	DD	54.85	49.20	52.03	50.50	46.50	52.03
20	*Discoglossus jeanneae*	DD	DD	DD	DD	DD	39.50	40.70	40.10	DD	DD	40.10
21	*Discoglossus montalentii*	1	DD	DD	DD	DD	DD	DD	60.00	DD	DD	60.00
22	*Discoglossus pictus*	1	22.05	9.65	15.85	DD	56.49	47.86	52.18	DD	DD	52.18
23	*Discoglossus sardus*	1	DD	DD	DD	DD	54.60	55.50	55.05	DD	DD	55.05
24	*Epidalea calamita*	1	46.81	53.63	50.22	DD	59.03	67.13	63.08	72.88	72.75	63.08

Telling R where to start

```
amphibians <- read_xlsx(
  "data/European_amphibians/oo_32985.xlsx",
  skip  = 3,
  sheet = 1,
  na    = "DD")
```

skip = 3: ignore metadata rows
sheet = 1: only use the first sheet
na = "DD": “DD” (data deficient) is used instead of NA

Metadata

Data about data

The information that describes your data:

- variable names,
- units,
- when and where collected

R has no system for this. Keeping it is on *you*.

Super important to do this well!

Column names are metadata

You need to strike a balance between **clarity** and **brevity**

Which of these examples would you prefer to be given?

larvdens

larval_density

not x

not larval_density_per_m3_in_ponds

Note: Use underscores or dots. *Never* spaces.

You can put metadata in the file

R ignores anything after #

You can use this to include metadata in a file, e.g.:

```
# Amniote traits, Myhrvold et al. 2015

# Missing values coded as -999
```

What we do in this course

Every folder in data/ has a README

- Source, licence, and any gotchas

Open data/Amniote_traits/README

Data styles

The same data can be represented in multiple different ways

Long/skinny/tidy format

One row per observation

station	date	species	seeds
1	1999-03-23	psd	5
1	1999-03-27	psd	5
2	1999-03-23	uva	5
2	1999-03-27	uva	5

Wide format

One row per station, dates as columns

station	species	03-23	03-27
1	psd	5	5
2	uva	5	5

Data Types in R

In R, all data have a type, e.g.:

- Numeric (2005)
- Logical (TRUE)
- Character (“Optimus Prime”)
- Factor (“Small” vs “Medium” vs “Large”)

Data Types in R

In R, all data have a type, e.g.:

- Numeric (2005)
- Logical (TRUE)
- Character (“Optimus Prime”)
- Factor (“Small” vs “Medium” vs “Large”)

To figure out what type something is, use `class()`

Data Types in R

Try `class()` out on a few things

```
class(“something”)

class(1)

class(1L)

class(TRUE)
```

Data Types in R

R tries to guess what type things are, but sometimes gets it very wrong!

- Numeric vs logical
- Date vs character or number
- Factor vs Numeric

These mistakes can cause big problems in analyses!

It’s important to check your data!

R read it. Is it right?

Try this (may require downloading from Github):

```
amniotes <- read.csv("data/Amniote_traits/
                      Amniote_Database_Aug_2015.csv")


mean(amniotes$litter_or_clutch_size_n)
mean(amniotes$longevity_y)
```

What do you get?

What happened?

This dataset codes missing values as −999

Try this:

```
amniotes <- read.csv(
      "data/Amniote_traits/Amniote_Database_Aug_2015.csv",
                  na.strings = -999)
mean(amniotes$litter_or_clutch_size_n, na.rm = TRUE)
```

We needed metadata to get this right!

The one that should worry you

Mean body mass, naive: 29,107 g

Mean body mass, correct: 37,493 g

Nothing about the first number looks wrong.

No error. No warning. Just wrong.

Higher data types

A single value is often not very useful, often we want sets of data:

- Vector
  - the most fundamental structure;
  - an ordered collection of elements of the same type
- Matrix
  - a 2D vector, all elements must be the same type
- Array
  - an n-dimensional generalization of a matrix

Higher data types

- List
  - ordered collection of objects that can be of different types or lengths
  - a very flexible container
- Data.frame
  - tabular data
  - essentially a list of equal-length vectors, often used for datasets
- Tibble
  - a modern variant of data.frame with cleaner printing and stricter rules

Higher data types

You can also use `class()` with these higher data types

Try this and see what class the avonet data you loaded earlier is

Accessing data within an object

For a vector or a list, you can access elements by their **position**:

```
test_vector <- c("Optimus Prime","Megatron")

test_vector[[1]]

test_vector[1]
```

Accessing data within an object

For a vector or a list, you can access elements by their **name**:

```
test_named_vector <- c("A" = "Optimus Prime",
                       "B" = "Megatron")

test_named_vector[["A"]]

test_named_vector["A"]
```

Accessing data within an object

For a vector or a list, you can access elements by their **name**:

```
test_named_vector <- c("A" = "Optimus Prime",
                       "B" = "Megatron")

test_named_vector[["A"]]

test_named_vector["A"]
```

You can find the names with the function `names()`

```
names(test_named_vector)
```

Accessing data within an object

For matrices, data.frames, and arrays there are more options!

- Rows (name or number)
- Columns (name or number)
- Row and column (name or number)

Same shape, different rules

Dataframes and matrices look similar, but work differently

- Data frame: names() or colnames()
- Matrix: colnames() only
- Data frame: x$a, x[["a"]], x[,"a"]
- Matrix: x[,"a"] only
- class(x) tells you which you have

Accessing data within an object

Let’s use the Avonet data from earlier

To access the species information, we can use:

```
avonet$Species1
```

```
avonet[1]
```

```
avonet[,1]
```

Try these out!

Accessing data within an object

Let’s use the Avonet data from earlier

To access the information for a particular row:

```
avonet[1,]
```

```
avonet["1",]
```
(note: only works because the row is named “1”)

Try these as well

Checking data within the avonet dataset

Use **`class()`** to check the data type

```
class(avonet$Species1)
```

```
class(avonet[,2])
```

Checking data with str()

Can check all the fields using the structure command, **`str()`**

```
str(avonet)
```

Try it out!

Checking data with str()

```
> str(avonet)
'data.frame':   11009 obs. of  37 variables:
 $ Sequence          : int  3103 3090 3125 3116 3092 3091 3089 3115 3130 3123 ...
 $ Species1          : chr  "Accipiter albogularis" "Accipiter badius" "Accipiter bicolor" "Accipiter brachyurus" ...
 $ Family1           : chr  "Accipitridae" "Accipitridae" "Accipitridae" "Accipitridae" ...
 $ Order1            : chr  "Accipitriformes" "Accipitriformes" "Accipitriformes" "Accipitriformes" ...
 $ Avibase.ID1       : chr  "AVIBASE-BBB59880" "AVIBASE-1A0ECB6E" "AVIBASE-ADBE44E1" "AVIBASE-68BF920B" ...
 $ Total.individuals : int  5 10 11 4 8 1 6 5 7 5 ...
 $ Female            : int  2 4 4 4 4 0 2 2 1 2 ...
 $ Male              : int  0 6 5 0 4 1 2 3 5 3 ...
 $ Unknown           : int  3 0 2 0 0 0 2 0 1 0 ...
 $ Complete.measures : int  4 8 8 3 4 1 4 4 6 4 ...
 $ Beak.Length_Culmen: num  27.7 20.6 25 22.5 21.1 20 20.5 19.2 20 25.4 ...
 $ Beak.Length_Nares : num  17.8 12.1 13.7 14 12.1 11.9 11.5 10.6 11.2 13.9 ...
 $ Beak.Width        : num  10.6 8.8 8.6 8.9 8.7 6.6 8.3 7.7 8.6 8.6 ...
 $ Beak.Depth        : num  14.7 11.6 12.7 11.9 11.1 12 10.9 9.6 11 13.2 ...
 $ Tarsus.Length     : num  62 43 58.1 61.2 46.4 48.7 52.6 60.3 43.6 62 ...
 $ Wing.Length       : num  235 187 230 202 218 ...
 $ Kipps.Distance    : num  81.8 62.5 56.6 64.1 87.8 42.9 38.9 81.3 49.5 77.8 ...
 $ Secondary1        : num  160 127 175 138 130 ...
 $ Hand.Wing.Index   : num  33.9 32.9 24.6 31.7 40.2 25.8 24 37.8 30 32.3 ...
 $ Tail.Length       : num  169 141 186 141 154 ...
 $ Mass              : num  249 131 288 142 186 ...
 $ Mass.Source       : chr  "Dunning" "Dunning" "Dunning" "Dunning" ...
 $ Mass.Refs.Other   : chr  NA NA NA NA ...
 $ Inference         : chr  "NO" "NO" "NO" "NO" ...
 $ Traits.inferred   : chr  NA NA NA NA ...
 $ Reference.species : chr  NA NA NA NA ...
 $ Habitat           : chr  "Forest" "Shrubland" "Woodland" "Forest" ...
 $ Habitat.Density   : int  1 2 2 1 1 1 1 1 2 ...
 $ Migration         : int  2 3 2 2 3 1 2 2 2 3 ...
 $ Trophic.Level     : chr  "Carnivore" "Carnivore" "Carnivore" "Carnivore" ...
 $ Trophic.Niche     : chr  "Vertivore" "Vertivore" "Vertivore" "Vertivore" ...
 $ Primary.Lifestyle : chr  "Insessorial" "Insessorial" "Generalist" "Insessorial" ...
```

Other quick ways to check objects

`str()` - gives info on object structure

`summary()` - provides summary information that varies by column class

`head()` - shows the first few rows of data

`table()` - used for categorical variables, shows how often combinations occur

`rownames()` - row names

`colnames()` - column names

Try these out!

What if R gets the type wrong?

You can convert between many different types using “as” commands

```
as.factor()

as.character()

as.data.frame()

as.matrix()
```

Dealing with NAs

You can remove any record that contains an NA using `na.omit()`

Some functions have arguments for handling NA values

- na.rm in `mean()`
- na.action in `lm()`
- use in `cor()`

You can turn the NA values into other values (e.g, 0)

- Only do this if it makes sense (e.g., number of species seen in a day)
- We'll discuss how to do this later

Next time:

Before class:

- read 2.4 - 2.5

During class:

- Discuss 2.4 - 2.5
- Discuss data we’re interested in
- Work through 2.6